Universidad Católica de Córdoba



"PROYECTO INTEGRADOR: ANALIZADOR SINTÁCTICO PARA MINIHTML"

Integrantes: ARTIGAU, Delfina, GRILLO, Facundo, GATTI, Pedro y LASA, Maria Emilia.

Materia: *"Lenguajes Formales y Autómatas"*.

Docente/s: *Lorena Bergamini*

Cátedra: A / A1

Carrera: Ingeniería en Informática

Fecha de Entrega: 03 / 11 / 2026

Introducción

Este informe presenta el diseño formal de un analizador sintáctico para MiniHTML, un subconjunto reducido de HTML en el que solo interesa la estructura de las etiquetas. El trabajo se organizó en tres etapas: una gramática libre de contexto que genera el lenguaje, un autómata a pila que lo reconoce y un programa en Python que simula ese autómata. Cada etiqueta se considera un único símbolo y el texto plano se representa con el token TEXTO.

Fase 1: Gramática libre de contexto

Definimos la gramática G = (V, Σ, S, P), donde:

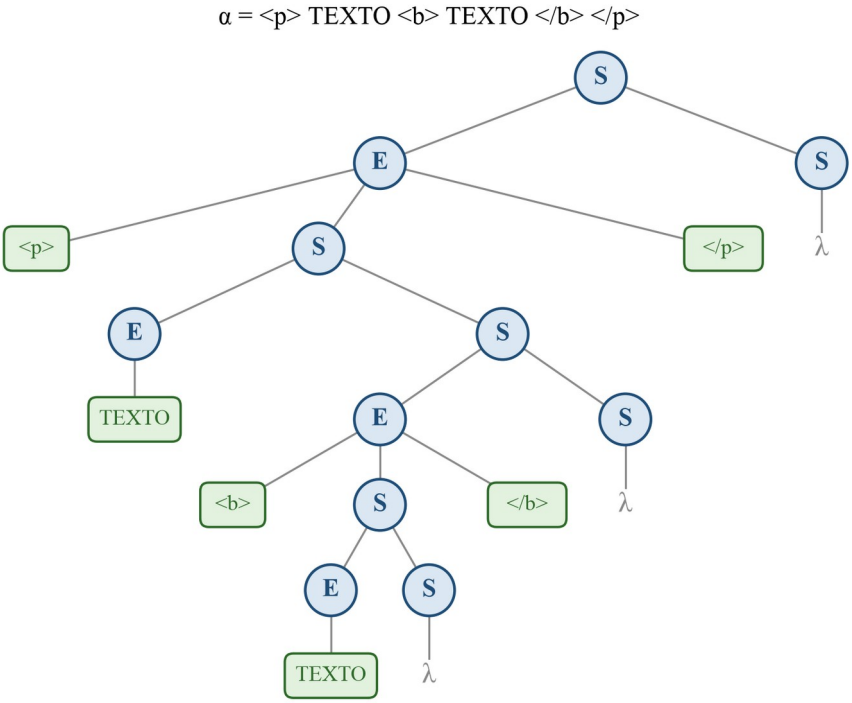
- V = { S, E }. S representa una secuencia de elementos y E representa un único elemento.
- Σ = { , , , , , , TEXTO }.
- S es el símbolo inicial.
- P es el conjunto de producciones:

N°	Producción
1	S → E S
2	S → λ
3	E → TEXTO
4	E → S
5	E → S
6	E → S

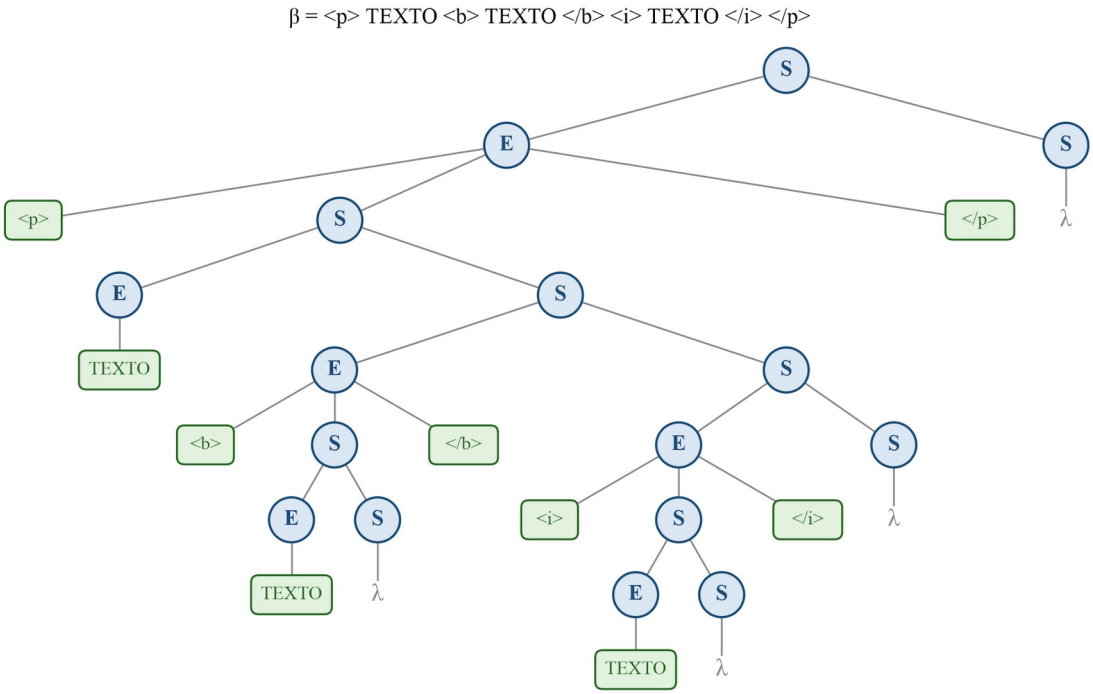
La apertura y el cierre de una etiqueta se generan en la misma producción, con S entre ambos. Por eso los cierres quedan siempre en orden inverso a las aperturas y no se puede derivar un cruce como   .

Separar S de E evita la ambigüedad: el próximo símbolo de la cadena determina la producción a aplicar, por lo que cada cadena tiene un único árbol de derivación. Además, S → λ permite el documento vacío y las etiquetas sin contenido, como .

***Árbol de derivación de α = TEXTO TEXTO ***



***Árbol de derivación de β = TEXTO TEXTO   TEXTO ***



Fase 2: Autómata a pila

Diseñamos un autómata a pila no determinista, con transiciones λ, que acepta por estado final: M = (Q, Σ, Γ, δ, q0, Z0, F), donde:

- Q = { q0, qf }.
- Σ = { , , , , , , TEXTO }.
- Γ = { Z0, B, I, P }. B, I y P indican que hay abierta una etiqueta , o .
- q0 es el estado inicial y Z0 es el símbolo de fondo de la pila.
- F = { qf }.
- δ: Q × (Σ ∪ {λ}) × Γ → P(Q × Γ*) es la función de transición de la tabla siguiente.

Cada etiqueta de apertura apila un símbolo que la identifica, el texto no modifica la pila y cada etiqueta de cierre desapila el símbolo del tope solo si corresponde a la misma etiqueta. Cuando termina la entrada y en la pila queda solo el marcador de fondo, una transición λ lleva al estado de aceptación.

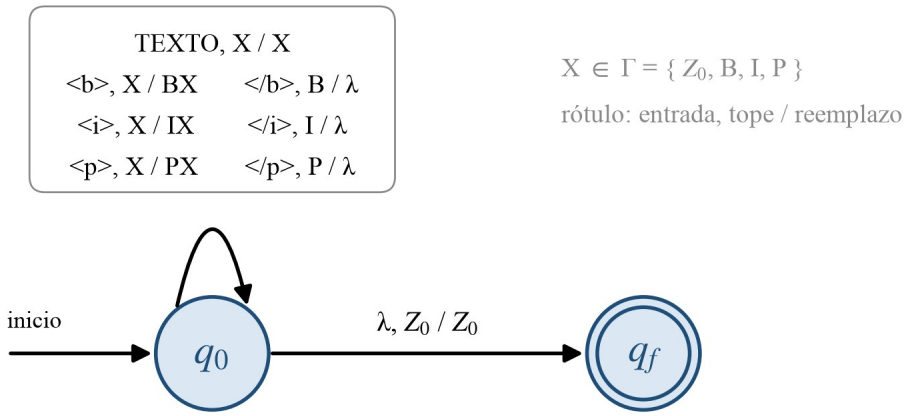
Función de transición

X representa cualquier símbolo de Γ, y en la pila el tope se escribe a la izquierda. Como cada δ(q, a, z) tiene a lo sumo un elemento, (q, γ) abrevia el conjunto {(q, γ)}.

N°	Estado	Entrada	Tope	δ(q, a, z)	Pila
1	q0	TEXTO	X	(q0, X)	Sin cambios
2	q0		X	(q0, BX)	Apila B
3	q0		X	(q0, IX)	Apila I
4	q0		X	(q0, PX)	Apila P
5	q0		B	(q0, λ)	Desapila B
6	q0		I	(q0, λ)	Desapila I
7	q0		P	(q0, λ)	Desapila P
8	q0	λ	Z0	(qf, Z0)	Acepta

En cualquier otro caso, δ(q, a, z) = ∅ y el autómata se detiene sin aceptar. El no determinismo aparece solo en q0 con Z0 en el tope, donde se puede leer un símbolo o aplicar la transición 8. Si se aplica antes de terminar la entrada, esa rama queda detenida en qf y no acepta.

Diagrama de transiciones



Trazas de configuraciones instantáneas

Cada configuración es (estado, entrada que falta leer, pila). A la derecha figura el número de la transición aplicada.

Cadena α = TEXTO TEXTO

(q0, TEXTO TEXTO , Z0)	inicial
⊢ (q0, TEXTO TEXTO , P Z0)	(4)
⊢ (q0, TEXTO , P Z0)	(1)
⊢ (q0, TEXTO , B P Z0)	(2)
⊢ (q0, , B P Z0)	(1)
⊢ (q0, , P Z0)	(5)
⊢ (q0, λ, Z0)	(7)
⊢ (qf, λ, Z0)	(8)

Cadena β = TEXTO TEXTO   TEXTO

(q0, TEXTO TEXTO TEXTO , Z0)	inicial
⊢ (q0, TEXTO TEXTO TEXTO , P Z0)	(4)
⊢ (q0, TEXTO TEXTO , P Z0)	(1)
⊢ (q0, TEXTO TEXTO , B P Z0)	(2)
⊢ (q0, TEXTO , B P Z0)	(1)
⊢ (q0, TEXTO , P Z0)	(5)
⊢ (q0, TEXTO , I P Z0)	(3)
⊢ (q0, , I P Z0)	(1)
⊢ (q0, , P Z0)	(6)
⊢ (q0, λ, Z0)	(7)
⊢ (qf, λ, Z0)	(8)

Cadena γ = TEXTO TEXTO   TEXTO

(q0, TEXTO TEXTO TEXTO , Z0)	inicial
⊢ (q0, TEXTO TEXTO TEXTO , P Z0)	(4)
⊢ (q0, TEXTO TEXTO , P Z0)	(1)
⊢ (q0, TEXTO , P Z0)	(1)
⊢ (q0, TEXTO , Z0)	(7)
⊢ (q0, TEXTO , I Z0)	(3)
⊢ (q0, , I Z0)	(1)
⊢ (q0, λ, Z0)	(6)
⊢ (qf, λ, Z0)	(8)

Las tres cadenas se consumen por completo y el autómata termina en qf, por lo que pertenecen a L(M).

Por qué un autómata finito no alcanza

Un autómata finito solo recuerda lo leído mediante su estado, y la cantidad de estados es fija. En MiniHTML el anidamiento no tiene límite y, para validar los cierres, hay que recordar cuántas etiquetas están abiertas y en qué orden se abrieron. Si se abren más párrafos seguidos que estados tiene el autómata, dos profundidades distintas lo dejan en el mismo estado y ya no puede saber cuántos cierres faltan. Por eso MiniHTML no es un lenguaje regular.

La pila resuelve el problema porque es una memoria sin límite y funciona en orden LIFO, igual que las etiquetas: la última que se abre es la primera que debe cerrarse. Al abrir una etiqueta se apila su símbolo, y al cerrarla se comprueba que ese símbolo esté en el tope.

Implementación

El reconocedor está implementado en Python 3, sin dependencias externas, en el archivo src/parser.py, y simula el autómata M. La función tokenizar convierte la entrada en tokens de Σ y la función simular aplica la tabla de transiciones usando una lista como pila. Ante una cadena inválida, el programa informa el motivo del rechazo: un cruce de etiquetas, un cierre sin apertura, etiquetas sin cerrar o un símbolo que no pertenece a Σ.

Conclusión

MiniHTML exige comprobar que las etiquetas se cierren en orden inverso al de apertura y sin límite de profundidad, algo que un autómata finito no puede hacer porque su memoria es acotada. La gramática G describe el lenguaje generando cada apertura junto con su cierre, y el autómata M lo reconoce guardando en la pila las etiquetas abiertas.